

**STRULE AUTOMATION**

Industrial Controls · Bay Area & Northern California

PLANT CHECKLIST

PLC / HMI / SCADA Migration Planning Checklist

Field notes from Jonathan Gilmour, independent controls engineer, Bay Area and Northern California.

This is the checklist I actually work from when a plant decides to pull an obsolete or unsupported controller and drop in a new one. It pairs with the longer blog post on migrating obsolete controllers; that post is the "why and how," this is the one-to-two page working document you print, mark up, and carry through the job. It's written to survive contact with a real plant, whether that's a crusher circuit on an aggregate site, a sort line at a materials recovery facility (MRF), or a digester and gas train at an anaerobic digestion (AD) / organics plant.

Rule of the whole exercise: **you are not allowed to know less about the plant after the migration than you did before it.** If a rung, an interlock, or a setpoint existed for a reason nobody wrote down, that reason is the job.

Safety header — read before any hands-on work

- ☐ **Lock-out/tag-out (LOTO) to a zero-energy state** before any hands-on work on the panel, field devices, or anything the panel drives. Verify zero energy at the point of work, don't assume the disconnect did it.
- ☐ **Energized-work caution:** if you must take live measurements (loop power, incoming control voltage, comparing states on a running machine), that's qualified-person, PPE, permit work. Put that step behind this warning every time — never bury an energized test inside a routine checklist line.
- ☐ **Stored energy on drives:** variable frequency drives (VFDs) and servo drives hold a charged DC bus after power-down. Wait out the manufacturer's discharge time (commonly a few minutes — check the drive's spec/label) and verify bus voltage is down before touching drive terminals.
- ☐ **Rotating / stored-energy equipment:** conveyors, screens, crushers, mixers, pumps, augers, and balers can restart or free-wheel. Block, chock, or pin as needed; a gravity take-up or a loaded incline conveyor will move when you release the brake.

- ☐ **Gas / hazardous-area note (AD, organics, biogas, digesters, sumps, pump vaults, bins/silos):** these are permit-required confined spaces. Hydrogen sulfide (H₂S) kills fast and deadens your sense of smell around ~100 ppm (IDLH ~100 ppm); methane is explosive from ~5% (lower explosive limit, LEL) to ~15% (upper explosive limit, UEL) in air. Test the atmosphere before entry, wear a bump-tested personal 4-gas monitor (H₂S, LEL, O₂, CO), and treat engulfment as real — never enter a bin or silo under a live draw. Biogas areas are typically Class I, Division 1/2 (or Zone 0/1/2) per NEC 500/505; any new device, gland, or wiring method in those areas has to keep that rating. Confirm the area classification drawing before you pick hardware.
 - ☐ **Never carry a defeated interlock forward.** If an interlock is jumpered, bypassed, or "temporarily" forced in the old system, that gets resolved and proven — not copied into the new code. More on this in commissioning.
-

1. Obsolescence and risk assessment

- ☐ List every controller, HMI, comms module, and I/O card in scope with make, model, firmware, and revision.
 - ☐ Vendor support status for each: active / mature / announced end-of-life / discontinued / unsupported. Get it from the vendor's lifecycle page, not memory.
 - ☐ Spare-parts reality: what's on the shelf here, what's still available new, what's grey-market/refurb only, what's unobtainium.
 - ☐ Lead time on a replacement for the single most critical card if it fails tomorrow. Write the number down. That number is your real risk.
 - ☐ Failure impact per system: what stops, for how long, and what it costs per hour (lost throughput, spoiled feedstock, gas flaring, permit/flow-compliance exposure, trucks turned away).
 - ☐ Single points of failure: one obsolete processor running the whole plant vs. one screen you can live without.
 - ☐ Programming-tool risk: can anyone still open, edit, and download the existing program? Do you have a working license, a machine old enough to run the software, and the right cable/driver? A controller you can't reprogram is already failed.
 - ☐ Prioritize: rank systems by (impact x likelihood x lead time). Migrate the worst first; don't let the easy one go first just because it's easy.
-

2. Capture the existing system BEFORE you touch it

This section is non-negotiable and comes before any hardware decision. You cannot map what you never wrote down.

- ☐ **Back up the running program** from every controller and HMI — upload the actual in-service copy, not last year's file from someone's laptop. Confirm it's the version that's running (compare,

don't trust the filename).

- ☐ Store backups in at least two places, dated, with firmware revision noted. Include the HMI/SCADA project, faceplates, scripts, and any recipe/parameter files.
 - ☐ **Document I/O** point-by-point: rack/slot/point, terminal, wire number, field device, signal type, range, engineering units, normal state.
 - ☐ **Networks:** draw what's actually there — protocols, IP addresses/node numbers, media, switches, serial links, remote I/O, radios, and any gateway/protocol-converter doing translation. Note baud rates and comm parameters on serial devices.
 - ☐ **Setpoints, ranges, tuning, and recipes:** capture PID gains, alarm limits, timers, counters, scaling, and every recipe/formula with its current values. On AD sites that means digester temperature bands, feed dosing/timing, mixing cycles, gas train pressures and flare/blend setpoints; on aggregate, belt-scale calibration, blend ratios, bin high/low levels; on MRF, line speeds, optical sorter and eddy-current settings, baler cycle parameters.
 - ☐ **As-built drawings:** find them, then verify them against the panel — as-builts lie. Redline what's wrong now, while the old system is still live to prove it.
 - ☐ **Undocumented logic and tribal knowledge:** walk the program with the people who run and maintain it. For every odd rung, force, delay, or workaround ask "why is this here?" and write the answer down. That 8-second start delay, that interlock bypass on a specific screen, that "we always run pump 2 first" — that's the knowledge that walks out the door with the tech who retires.
 - ☐ Note every existing **force, bypass, and jumper** and the story behind it. These get resolved before or during migration, never blindly reproduced.
 - ☐ Photograph panel interiors, nameplates, and field devices. Capture HMI screens as they run.
-

3. Scope and I/O mapping

- ☐ Fix the boundary: exactly which panels, screens, drives, instruments, and network segments are in this migration and which are staying. Write down the interfaces to everything staying.
- ☐ Build a **point-for-point I/O map**: legacy address -> new tag, with signal type, range, units, and field device on each line. No orphan points; every old point either maps forward or is deliberately retired with a reason.
- ☐ Legacy addressing to modern tag names: adopt a readable, consistent tag convention now (area/equipment/function). Keep a cross-reference back to old addresses for the first year of troubleshooting.
- ☐ **Obsolete field devices:** flag sensors, analyzers, and drives that are themselves end-of-life or speak a dying protocol. Decide device-by-device: keep, replace now, or plan to replace. Don't strand a brand-new PLC behind a gateway to a dead instrument.

- ☐ Confirm I/O count and type headroom on the new platform (analog/digital/high-speed/specialty like weigh, RTD, thermocouple). Leave spare points; you will need them.
 - ☐ Verify power, grounding, and panel space/heat for the new hardware — a modern processor and its I/O may not drop into the old footprint.
-

4. Platform selection (vendor-neutral)

- ☐ Judge platforms on fit, not logo. Consider comms compatibility, tool availability, in-house familiarity, support presence in your region, and total lifecycle — not just purchase price.
 - ☐ **Standardize on what your team can actually support.** One platform the plant already knows, with parts on the shelf, beats a marginally better one nobody can troubleshoot at 2am.
 - ☐ Spares and cross-plant commonality: if you run three sites, having the same processor and I/O family across them shrinks your spares pool and your training load.
 - ☐ **Comms compatibility:** confirm the new controller talks natively to what's staying — drives, weigh systems, gas analyzers, optical sorters, SCADA, plant historian, and any corporate/network layer. Native protocol beats a gateway; count every gateway you'd need and treat each as a new failure point and a new thing to document.
 - ☐ Cybersecurity and network posture: segmentation, remote-access method, and patchability. Don't put a fresh internet-reachable hole in the plant network in the name of "remote support."
 - ☐ Licensing: know what runtime, tag-count, and development licenses you'll own, and that you can still open this project in 10 years.
 - ☐ Hazardous-area fit (AD/biogas): any hardware going into a classified area, or wiring that crosses into one, has to hold the Class/Division (or Zone) rating on the area drawing.
-

5. Code approach — like-for-like vs. clean rewrite

- ☐ Decide, and write down why: **like-for-like translation** (fastest, lowest risk, carries old baggage) vs. **clean rewrite** (better structure, higher effort and test burden). A mix is common: rewrite the messy subsystem, translate the rest.
- ☐ **Interlock and alarm parity is mandatory** either way. Every protective interlock and every alarm in the old system is accounted for in the new one — same trip conditions, same or better — unless a deliberate, recorded, engineered decision changes it. Build an interlock/alarm matrix now and use it as your proving list at SAT.
- ☐ Improve deliberately, not by accident: better tag names, cleaner alarm rationalization, real diagnostics, first-out fault indication, consistent faceplates. Log each intentional change.
- ☐ Leave alone what works and isn't understood yet. If nobody can explain a rung's purpose, do not "clean it up" — replicate it and flag it for investigation. Silent behavior changes are how migrations hurt people.

- ☐ Resolve every legacy force/bypass here on paper: for each, decide fix-the-root-cause vs. legitimate retune vs. genuinely-needed-and-documented. If a nuisance setpoint was being defeated, the fix is retuning it inside the equipment's rated limit — one change at a time, recorded — not carrying the jumper forward. Defeating a protective function or gas detection is never the answer.
 - ☐ Version-control the new code with real commit notes. Keep the translation cross-reference (old address/rung -> new tag/routine).
-

6. Factory Acceptance Test (FAT)

- ☐ Test the new system on the bench before it ever sees the plant. Simulate I/O (sim cards, a test harness, or a soft simulator) and drive every input.
 - ☐ Prove every interlock and alarm from the matrix in Section 5 — trip it, confirm the reaction, confirm first-out and reset behavior.
 - ☐ Exercise sequences, modes, permissives, and failure responses (loss of comms, loss of a sensor, e-stop, power cycle). Confirm safe-state on fault.
 - ☐ Test HMI/SCADA screens against the logic: navigation, faceplates, alarm presentation, security levels, and any scripting.
 - ☐ Verify comms to a real or simulated instance of each external system (drive, historian, gas analyzer, sorter).
 - ☐ Run recipes/parameters through and confirm values land correctly and scale right.
 - ☐ Write a FAT punch list. Nothing goes to site with an open safety item. Get sign-off before it ships.
-

7. Cutover planning

- ☐ Define the **downtime window** honestly, with margin. Coordinate with production, feedstock deliveries, gas offtake/flare permits, and outbound commitments. AD plants especially: the biology and the gas train don't pause cleanly — plan digester and gas-handling state through the outage.
- ☐ **Staged vs. big-bang**: stage it if you can (subsystem by subsystem, or run new alongside old on non-critical points first). Big-bang only when the plant truly can't run split — and then your rollback plan has to be airtight.
- ☐ **Written ROLLBACK plan**. Before you pull the first wire: exactly how you put the old system back, who decides to abort, the go/no-go decision points and times, and the state you leave the plant in if you abort mid-cutover. Rehearse the decision, not just the hope.
- ☐ **Keep the old system recoverable**. Do not scrap, wipe, or gut the old controller until the new one has run clean through commissioning and a shakeout period. Keep the old program, old

cards, and old wiring documented and physically re-installable. Label removed wires; don't cut to length until you're sure.

- ☐ Pre-stage everything: hardware built and powered-checked, firmware matched, spares on hand, cables made, laptops/licenses ready, printed I/O map and drawings on the table.
 - ☐ LOTO plan for the cutover written and briefed. Everyone knows what's locked, who holds the key, and the sequence for re-energizing.
 - ☐ Contingency: who to call, how long parts take, and the "we're staying down past the window" trigger and message.
-

8. Data, historian, and recipe migration

- ☐ Map historian/SCADA tags old -> new and decide how history is preserved: keep the archive read-only, backfill, or bridge tag names so trends don't go blank.
 - ☐ Migrate recipes/formulas/parameter sets and **verify each value against the captured original** (Section 2). Don't retype from memory; diff against the backup.
 - ☐ Reproduce reports, compliance logs, and regulatory records (flow, emissions, gas, throughput) — confirm they still populate correctly. On AD/organics, regulatory reporting continuity matters; a gap in the record is a finding.
 - ☐ Preserve alarm history and event logs enough to investigate anything the migration itself might cause.
 - ☐ Confirm time sync (NTP) across controllers, HMIs, and historian so trends and event order actually line up.
-

9. Commissioning and Site Acceptance Test (SAT)

- ☐ LOTO and safe-state governs the whole commissioning. Bring equipment up deliberately, one system at a time, permissives first.
- ☐ **Rotating-equipment caution before you stroke outputs:** stroking an output moves the real device — a conveyor, screen, crusher, auger, mixer, or pump can start. Clear and control personnel around anything that can move, brief a spotter, and confirm nobody is in a pinch point or under a take-up before you energize the output. Never stroke an output blind.
- ☐ **Loop-check every I/O point** end to end at the plant: stroke each output to the real device, exercise each input from the real sensor, confirm the tag, the scaling, and the direction. This is where map errors surface.
- ☐ **Cross-reference and physically prove every interlock and safety function** against the matrix — trip each one for real and confirm the equipment does the safe thing. This is the single most important step in the whole migration.

- ☐ **Never carry a defeated interlock forward.** If a protective function can't be proven working, the plant does not run in that mode. A genuine nuisance trip gets retuned inside the rated limit, recorded, one change at a time — it does not get bypassed. Gas detection and safety shutdowns are proven live, not assumed.
 - ☐ Prove protective functions after migration: e-stops, overtravel/overspeed, high-level and overflow, high-temp, low-flow, gas detection and gas-train shutdowns, drive faults, pull-cords on conveyors. Test them; don't infer them.
 - ☐ Run the process through normal, upset, and shutdown conditions. Watch first-out, alarm behavior, and safe-state on induced faults (yank a sensor, drop comms).
 - ☐ SAT punch list, dated, with owner and sign-off. Keep the old system recoverable until the punch list is closed and the plant has run a clean shakeout.
-

10. Operator training, documentation, and handover

- ☐ Train operators and maintenance on the new HMI/SCADA before you hand over — new screens, new alarm behavior, new navigation, and what changed from the old system.
 - ☐ Update **as-builts to reflect what was actually installed**, including the redlines you found in Section 2. The new drawings are the ones that must be true.
 - ☐ Deliver the documentation set: annotated program, I/O map with old/new cross-reference, network diagram, interlock/alarm matrix with proof records, setpoint/recipe list, backups (controller + HMI), and firmware revisions.
 - ☐ Capture the tribal knowledge you collected as written notes attached to the code and drawings — so the next person doesn't have to re-earn it.
 - ☐ Record what changed and why (the deliberate improvements from Section 5) so future troubleshooting starts from truth.
 - ☐ Confirm the plant can back up, restore, and edit the new system itself: tools, licenses, cables, and a named person who knows how.
 - ☐ Archive the old system's final backups and keep them recoverable for a defined shakeout period before anything gets scrapped.
-

A closing note, plainly: an independent second set of eyes earns its keep most on the faults that get handed between vendors — where the old integrator is gone, the drawings don't match, and nobody left can say why a rung is there. That's exactly the ground a migration crosses. If you've captured the plant honestly before you touched it (Section 2), you'll rarely need anyone; if you didn't, that's the call to make early, not at 2am on cutover night.

Numbers here are typical/approximate and framed to check against your own equipment's spec and your area classification drawings. Nothing in this document authorizes defeating a protective interlock, safety function, or gas detection.